DevOps Engineer Technical Assignment Report

1. Project Overview

Objective

The objective of this project was to deploy a production-like web application on AWS Free Tier using DevOps best practices. The project includes automated deployment, monitoring, logging, security configuration, and performance testing.

Application

A simple Node.js and Express.js web application was developed using the EJS template engine. The application includes:

- Home page
- Application information API (`/api/info`)
- Health check endpoint (`/health`)

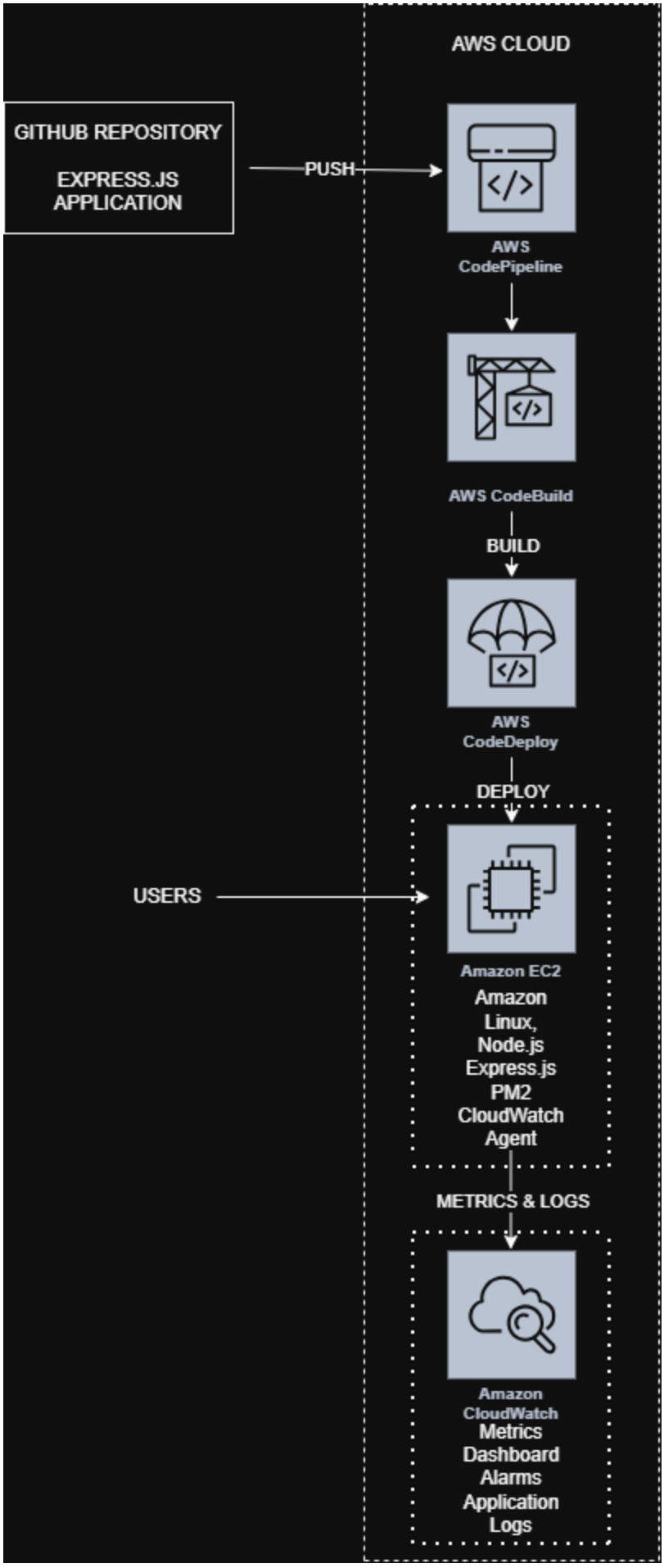
The application was deployed on an Amazon EC2 instance.

---

2. Architecture

The solution consists of the following AWS services:

- Amazon EC2
- AWS IAM
- AWS CodePipeline
- AWS CodeBuild
- AWS CodeDeploy
- Amazon CloudWatch


AWS CLOUD
GITHUB REPOSITORY
EXPRESS.JS APPLICATION
PUSH
AWS CodePipeline
AWS CodeBuild
BUILD
AWS CodeDeploy
DEPLOY
USERS
Amazon EC2
Amazon Linux, Node.js Express.js PM2 CloudWatch Agent
METRICS & LOGS
Amazon CloudWatch
Metrics Dashboard Alarms Application Logs

---

3. AWS Infrastructure

The following resources were created for this project.

Service	Purpose
Amazon EC2	Hosts the Express.js application
IAM Role	Provides secure permissions for EC2
Security Group	Allows HTTP traffic on port 3000 and SSH access
CodePipeline	Automates the CI/CD process
CodeBuild	Builds the application
CodeDeploy	Deploys the application to EC2
CloudWatch	Monitoring, logging, dashboard, and alarms

---

4. Application Deployment

The application source code was stored in a GitHub repository.

An EC2 instance running Amazon Linux was launched, and Node.js, npm, PM2, and the CodeDeploy agent were installed.

The application was managed using PM2 to ensure that it remained running even after deployment or system restart.

The application was successfully accessed through the EC2 public IP address on port 3000.

---

5. CI/CD Pipeline

A complete CI/CD pipeline was created using AWS services.

The deployment process is as follows:

1. Developer pushes code to GitHub.
2. CodePipeline detects the changes.
3. CodeBuild installs project dependencies and prepares the deployment package.
4. CodeDeploy deploys the application to the EC2 instance.
5. PM2 restarts the application automatically.

The deployment process is fully automated and requires no manual intervention after code is pushed to GitHub.

---

6. Security

Basic security best practices were implemented.

- IAM roles were used instead of storing AWS credentials on the server.
- Least privilege permissions were assigned wherever possible.
- Security Groups were configured to allow only the required inbound traffic.
- The Express application uses Helmet to add common HTTP security headers.
- Application logs and metrics are collected securely using the CloudWatch Agent.

---

7. Monitoring and Logging

Amazon CloudWatch was configured for monitoring.

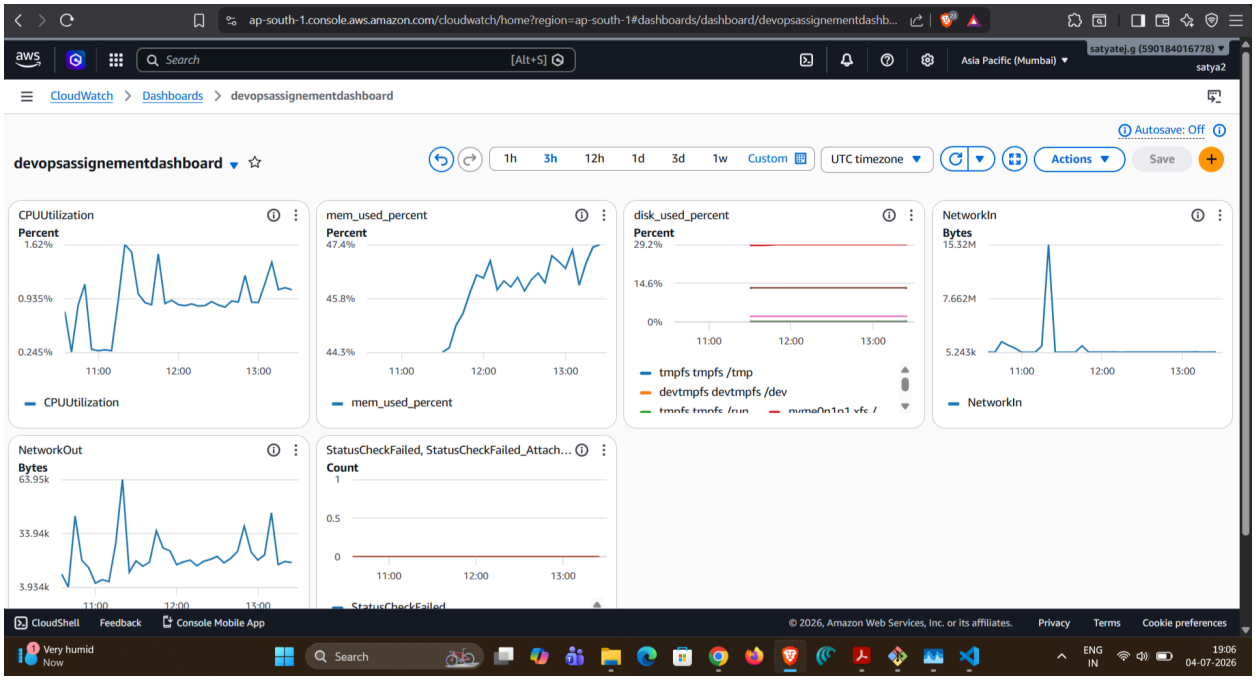
The following metrics were collected:

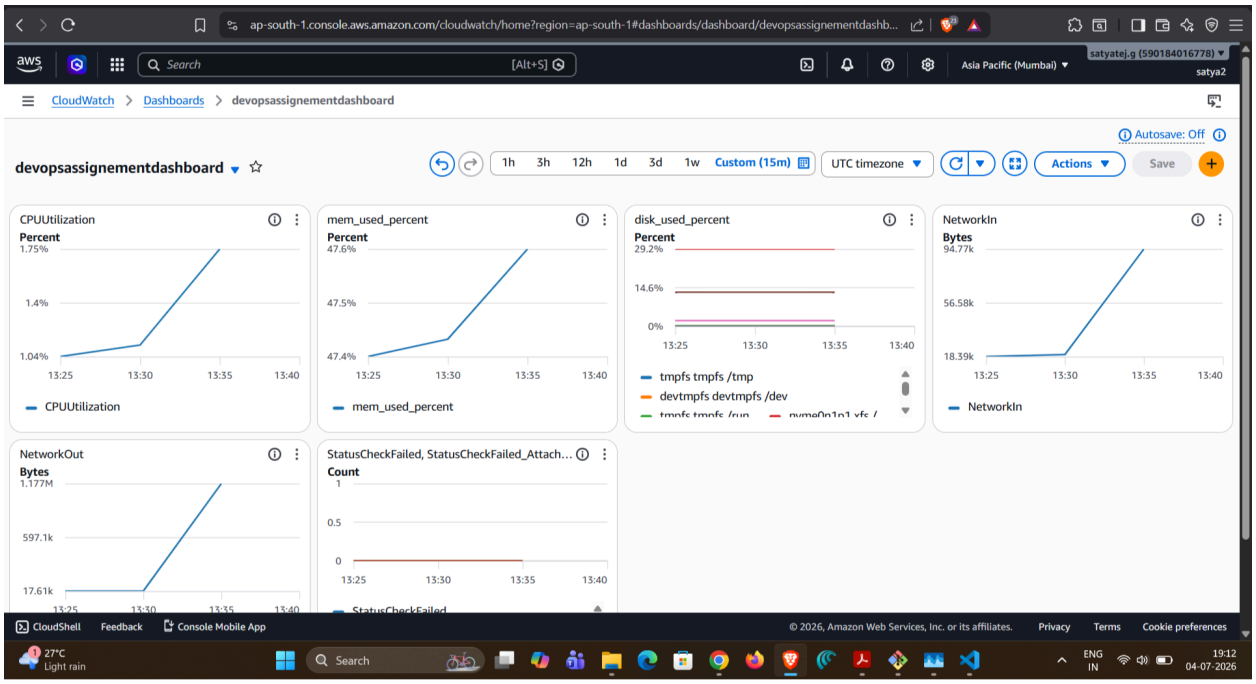
- CPU Utilization
- Memory Utilization
- Disk Usage
- Disk I/O

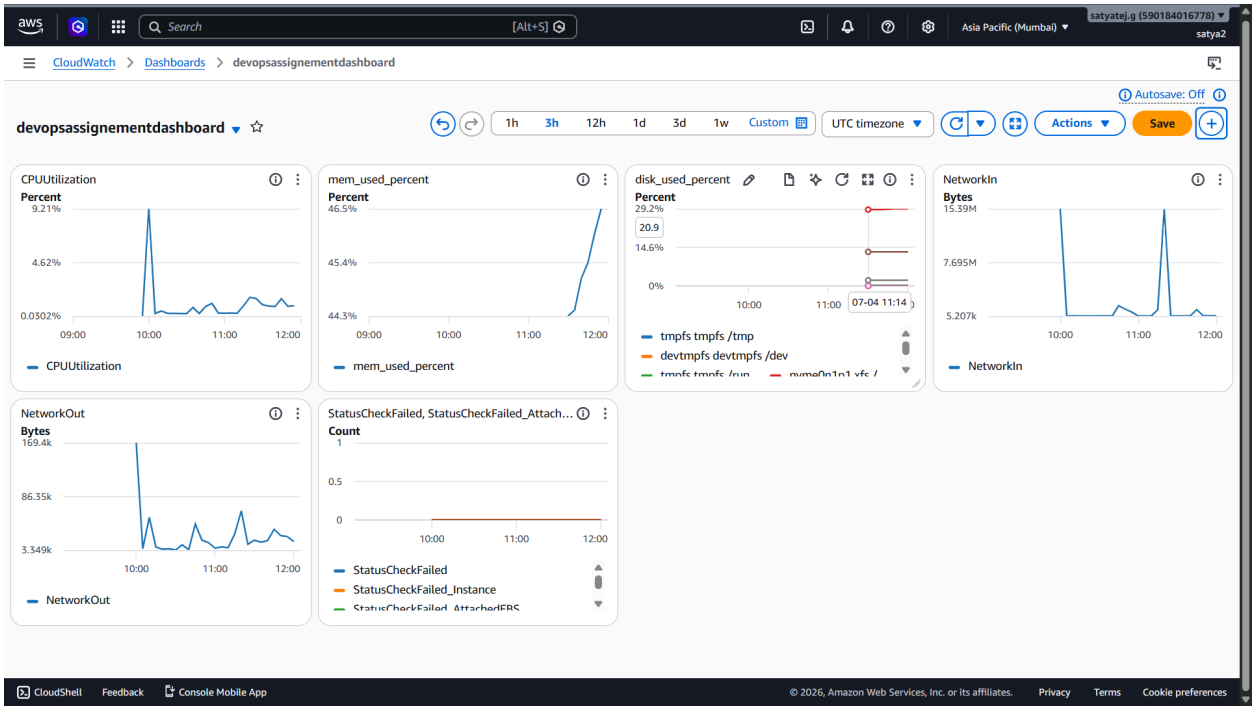
A CloudWatch Dashboard was created to monitor the EC2 instance in real time.

A CloudWatch Alarm was configured to notify when CPU utilization exceeds the configured threshold.

Application logs generated by PM2 were collected using the CloudWatch Agent and stored in CloudWatch Log Groups.


ap-south-1.console.aws.amazon.com/cloudwatch/home?region=ap-south-1#dashboards/dashboard/devopsassignementdashb...
Search
[Alt+S]
Asia Pacific (Mumbai)
satyatej.g (590184016778)
satya2
CloudWatch
Dashboards
devopsassignementdashboard
Autosave: Off
devopsassignementdashboard
1h
3h
12h
1d
3d
1w
Custom
UTC timezone
Actions
Save
CPUUtilization
Percent
1.62%
0.935%
0.245%
11:00
12:00
13:00
CPUUtilization
mem_used_percent
Percent
47.4%
45.8%
44.3%
11:00
12:00
13:00
mem_used_percent
disk_used_percent
Percent
29.2%
14.6%
0%
11:00
12:00
13:00
tmpfs tmpfs /tmp
devtmpfs devtmpfs /dev
NetworkIn
Bytes
15.32M
7.662M
5.243k
11:00
12:00
13:00
NetworkIn
NetworkOut
Bytes
65.95k
33.94k
3.934k
StatusCheckFailed, StatusCheckFailed_Attach...
Count
1
0.5
0
11:00
12:00
13:00
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
Very humid
Now
Search
ENG
IN
19:06
04-07-2026


ap-south-1.console.aws.amazon.com/cloudwatch/home?region=ap-south-1#dashboards/dashboard/devopsassignementdashb...
Search
[Alt+S]
Asia Pacific (Mumbai)
satyatej.g (590184016778)
satya2
CloudWatch
Dashboards
devopsassignementdashboard
Autosave: Off
devopsassignementdashboard
1h
3h
12h
1d
3d
1w
Custom (15m)
UTC timezone
Actions
Save
CPUUtilization
Percent
1.75%
1.4%
1.04%
13:25
13:30
13:35
13:40
CPUUtilization
mem_used_percent
Percent
47.6%
47.5%
47.4%
13:25
13:30
13:35
13:40
mem_used_percent
disk_used_percent
Percent
29.2%
14.6%
0%
13:25
13:30
13:35
13:40
tmpfs tmpfs /tmp
devtmpfs devtmpfs /dev
NetworkIn
Bytes
94.77k
56.58k
18.39k
13:25
13:30
13:35
13:40
NetworkIn
NetworkOut
Bytes
1.177M
597.1k
17.61k
StatusCheckFailed, StatusCheckFailed_Attach...
Count
1
0.5
0
13:25
13:30
13:35
13:40
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
27°C
Light rain
Search
ENG
IN
19:12
04-07-2026



---

8. Load Testing

Performance testing was performed using k6 from a local machine.

The following endpoints were tested:

- `/`
- `/api/info`

The following performance metrics were recorded:

- Response Time
- Latency
- Throughput
- Error Rate
- CPU Utilization
- Memory Utilization

CloudWatch metrics were monitored while the tests were running.

---

9. Performance Analysis

The application remained stable throughout the load test.

Observations:

- CPU utilization increased slightly during testing.
- Memory usage remained stable.
- Response times remained low.
- No request failures were observed.
- The application successfully handled the generated workload.

Since the application is lightweight and runs on a single EC2 instance, resource utilization remained relatively low.

---

10. Challenges Faced

During the implementation, a few issues were encountered.

CloudWatch Logs

The CloudWatch Agent could not access the PM2 log files because of Linux file permissions.

The issue was resolved by updating the directory permissions so that the CloudWatch Agent user could read the log files.

CodeDeploy

The first deployment failed because a manually created application directory already existed on the EC2 instance.

The issue was resolved by removing the manually created directory and allowing CodeDeploy to manage the deployment.

PM2

An "Address already in use" error occurred because another Node.js process was already using port 3000.

The issue was resolved by stopping the manually started process and allowing PM2 to manage the application.

---

11. Future Improvements

The following improvements can be made in a production environment:

- Configure HTTPS using Nginx and SSL certificates.
- Add an Application Load Balancer.
- Configure Auto Scaling.
- Host static assets using Amazon S3.
- Provision infrastructure using Terraform or AWS CloudFormation.
- Containerize the application using Docker and deploy it to Amazon ECS or Amazon EKS.

---

12. Conclusion

This project successfully demonstrated the deployment of a production-like Node.js application on AWS using DevOps practices.

A complete CI/CD pipeline was implemented using AWS CodePipeline, CodeBuild, and CodeDeploy. Monitoring, logging, and alerting were configured using Amazon CloudWatch, and performance testing was completed using k6.

The project demonstrates practical knowledge of AWS services, deployment automation, monitoring, security, and basic performance testing using AWS Free Tier resources.